Initiation au développement : Variables, Structures de Contrôle et Modularité

Thème principal : Bases de la programmation impérative, gestion de la mémoire et algorithmique

Synthèse exécutive : Ce cours pose les fondements indispensables de la programmation impérative en R1.01. Il enseigne la gestion rigoureuse de la mémoire RAM via le typage strict des variables, l'arbitrage entre les structures de contrôle (instructions conditionnelles et choix entre boucles POUR/TANT QUE) pour piloter le flux d'exécution, et la décomposition modulaire au moyen de fonctions et procédures. L'objectif est d'apprendre à écrire des algorithmes performants, lisibles, sans boucles infinies ni effets de bord imprévus.

1. Les Fondements des Variables, Typage Fort et Allocation Mémoire

Une variable n'est pas un simple concept abstrait, c'est une adresse physique exacte dans la mémoire vive (RAM) du système. Lors de la 'déclaration', le compilateur ou l'interpréteur réserve une plage précise d'octets en fonction du type spécifié (par exemple 4 octets pour un entier standard, 8 octets pour un double flottant). Le typage fort empêche la corruption de données en refusant l'écriture de bits incohérents dans cet espace. L'affectation consiste à écrire une valeur binaire à cette adresse mémoire. Si une variable est lue avant son initialisation explicite, le programme lira les 'poubelles mémoires' (des résidus binaires laissés par d'autres applications), provoquant des comportements aléatoires ou imprévisibles.

- Variable et Adresse RAM : Un espace réservé en mémoire vive identifié par un nom et un type.
- Déclaration vs Affectation : Déclarer réserve le bloc d'octets ; affecter modifie son contenu binaire.
- Typage Fort : Définit la taille exacte en octets et interdit les opérations invalides entre types incompatibles.
- Résidus Binaires (Garbage Values) : Risque de lire une variable non initialisée contenant d'anciennes données.

Cas concret enseignant : *Le professeur compare la variable à une boîte aux lettres de taille fixe. Si l'on tente d'y insérer une valeur incompatible (comme un nombre à virgule dans un emplacement prévu pour un entier), le système tronque brutalement la partie décimale ou génère une erreur d'exécution.*

2. Pilotage du Flux d'Exécution : Conditionnelles et Arbitrage des Boucles

Les structures de contrôle dirigent la séquence dans laquelle les instructions sont exécutées. L'aiguillage conditionnel évalue une expression logique vraie ou fausse pour exécuter la branche correspondante. Concernant la répétition, le choix de la boucle est crucial : la boucle POUR est préconisée lorsque le nombre exact d'itérations est connu d'avance (ex: parcourir un tableau de taille N). La boucle TANT QUE est indispensable lorsque l'arrêt dépend d'un événement externe ou d'une valeur calculée pendant l'exécution. Dans un TANT QUE, la variable participant à la condition de test DOIT impérativement être modifiée dans le corps de la boucle, sous peine d'enfermer le programme dans une boucle infinie qui bloque le processeur.

- Aiguillage Conditionnel (SI... ALORS... SINON) : Orientation du code selon l'évaluation d'une expression booléenne.
- Boucle POUR (FOR) : Répétition à compteur automatique pour un nombre d'itérations connu à l'avance.
- Boucle TANT QUE (WHILE) : Répétition subordonnée à une condition dynamique évaluée avant chaque itération.
- Boucle Infinie : Plantage causé par l'absence d'évolution de la condition d'arrêt.

Cas concret enseignant : *L'enseignant illustre la boucle TANT QUE avec le guichet automatique bancaire (DAB) : la machine vous demande de saisir votre code PIN TANT QUE le code est faux ET QUE le nombre d'essais reste strictement inférieur à 3.*

3. Modularité : Fonctions, Portée (Scope) et Passage de Paramètres

La modularité permet de respecter le principe de séparation des responsabilités. Une fonction est une boîte noire qui prend des arguments en entrée, effectue un traitement interne et renvoie une valeur unique grâce à l'instruction RETURN. Une procédure ne renvoie rien. La portée des variables garantit qu'une variable déclarée à l'intérieur d'une fonction (variable locale) est allouée sur la pile d'exécution puis détruite dès la fin de la fonction. Lorsqu'on passe un paramètre par VALEUR, l'algorithme duplicate la valeur dans un nouvel emplacement mémoire ; toute modification reste locale. Lors d'un passage par RÉFÉRENCE, l'adresse mémoire exacte est transmise : la fonction modifie directement la variable de l'appelant.

- Modularité : Découpage d'un programme complexe en sous-programmes autonomes et réutilisables.
- Fonction vs Procédure : Une fonction retourne obligatoirement une valeur via RETURN ; une procédure réalise une tâche sans retour explicit.
- Portée (Scope) : Durée de vie et visibilité limitée d'une variable au sein de son bloc ou sous-programme.
- Passage par Valeur vs Référence : Travail sur une copie locale (valeur) vs manipulation directe de l'adresse mémoire (référence).

Cas concret enseignant : *Analogie de la recette de cuisine : transmettre des arguments par valeur équivaut à donner une photocopie de sa recette à un ami (s'il gribouille dessus, votre original reste intact) ; les passer par référence équivaut à lui donner l'original dans votre propre cahier (toute annotation modifie définitivement l'unique recette).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi l'indice d'un tableau commence-t-il à 0 et pas à 1 en informatique ?
Réponse du professeur : L'indice ne représente pas le rang de l'élément, mais le 'décalage' (l'offset) par rapport à l'adresse mémoire du tout premier élément. T[0] signifie 'adresse de départ + 0 octet', donc le premier élément. T[1] signifie 'adresse de départ + 1 fois la taille du type'.

Question étudiant : Que se passe-t-il si on oublie de faire évoluer la variable de contrôle dans une boucle TANT QUE ?
Réponse du professeur : La condition d'arrêt restera toujours vraie. Le programme entre dans une 'boucle infinie', consomme 100% des ressources du coeur processeur qui lui est alloué et se bloque sans jamais restituer la main.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme complet qui parcourt un tableau T de N entiers, recherche la valeur maximale présente et retourne son indice.

Résolution pas à pas :
1. S'assurer que le tableau n'est pas vide (N > 0). 2. Initialiser une variable locale 'indice_max' à 0 (on suppose que le premier élément est provisoirement le plus grand). 3. Initialiser une variable locale 'valeur_max' avec T[0]. 4. Utiliser une boucle POUR avec un compteur 'i' allant de 1 jusqu'à N - 1 : a. SI T[i] > valeur_max ALORS : i. valeur_max <- T[i] ii. indice_max <- i 5. À la sortie de la boucle, retourner 'indice_max'.

Résultat : L'algorithme s'exécute en une seule passe linéaire (complexité temporelle O(N) et spatiale O(1)). Il retourne l'index exact de la première occurrence de la valeur maximale sans altérer le tableau.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale entre une fonction et une procédure ?

R : Une fonction calcule et renvoie obligatoirement une valeur au programme appelant via l'instruction RETURN, tandis qu'une procédure exécute des actions sans retourner de valeur.

Q : Pourquoi est-il dangereux de lire la valeur d'une variable non initialisée ?

R : Parce qu'elle contient des résidus binaires laissés par d'autres processus dans la mémoire RAM ('mémoire poubelle'), provoquant un comportement imprévisible de l'algorithme.

Q : Dans quel cas faut-il privilégier une boucle POUR plutôt qu'une boucle TANT QUE ?

R : On privilégie la boucle POUR lorsque le nombre d'itérations est connu à l'avance avant d'entrer dans la boucle.